

ECE 405 Assignment 1

Justin Kugiyama

February 2026

Link to Github Repo: ECE Assignment 1

Written Responses

Unicode 1

- a) The function `chr(0)` returns the null Unicode character with code point U+0000.
- b) This character's string representation differs from its printed representation because the string representation appears as the escape sequence `\x00`, while printing it produces no visible output since the null character is non-printing.
- c) When this character occurs in text, nothing appears in its place. It does not act as a space; rather, it behaves as an invisible control character.

Unicode 2

- a) It is preferred to train a tokenizer on UTF-8 rather than UTF-16 or UTF-32 because UTF-8 uses 1-byte code units for ASCII characters, while UTF-16 uses 2 bytes and UTF-32 uses 4 bytes per code unit. This makes UTF-8 more space-efficient in practice. Additionally, UTF-8 is backward compatible with ASCII, which is dominant in web data.

b)

```
def decode_utf8_bytes_to_str(bytestring: bytes) -> str:
    return bytestring.decode("utf-8")
```

```
print(decode_utf8_bytes_to_str("hello".encode("utf-8")))
```

- c) The function decodes a byte string using UTF-8. However, decoding each byte separately would fail for non-ASCII UTF-8 sequences because multi-byte characters must be interpreted together. The two-byte sequence `0xC0 0xAF` does not decode to any Unicode character because it is an invalid overlong UTF-8 encoding.

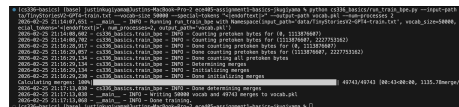


Figure 1: BPE Training on TinyStories

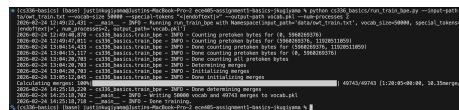


Figure 2: BPE Training on OWT

BPE Tokenizer Training

BPE Training on TinyStories

a) Training the byte-level BPE tokenizer on TinyStories took about 3.5 minutes, with peak memory usage well under 30 GB. The longest token in the vocabulary was an unusually long repeated or multi-syllable string. This makes sense because byte-level BPE merges frequent substrings, and repetitive child-story text can lead to long merged tokens.

b) Profiling shows that counting pretoken byte frequencies during preprocessing dominates runtime, while the merge computation itself is relatively fast once frequencies are known.

BPE Training on OWT

(a) Training the 32,000-vocab byte-level BPE tokenizer on OpenWebText took about 1 hour and 35 minutes total. The longest token was a very long web-related substring (such as part of a URL, HTML pattern, or repeated formatting text), which makes sense because OpenWebText contains a lot of repeated web boilerplate that BPE merges into single tokens.

(b) Compared to TinyStories, the OpenWebText tokenizer learns more technical and web-specific tokens, while the TinyStories tokenizer mainly learns simple narrative words and common English phrases, resulting in cleaner and more story-focused tokens.

Implementing the Tokenizer

This code implements a Byte Pair Encoding (BPE) tokenizer that converts text into token IDs and back. During encoding, it optionally preserves special tokens, splits text via regex, converts characters to UTF-8 bytes, and repeatedly applies

learned merge rules to combine frequent byte pairs into subword tokens. These byte sequences are mapped to integer IDs. During decoding, token IDs are converted back to bytes, concatenated, and decoded into a UTF-8 string.

Experiments with Tokenizers

a) The OpenWebText 32K tokenizer achieves a higher compression ratio (more bytes per token) than the TinyStories 10K tokenizer because its larger vocabulary captures longer and more diverse subword patterns, producing fewer tokens per document.

b) When tokenizing OpenWebText with the TinyStories tokenizer, compression decreases because the smaller 10K vocabulary cannot efficiently represent broader web vocabulary, resulting in more fragmented subwords.

c) If processing occurs at roughly 5 MB/s, tokenizing the 825GB Pile dataset would take approximately 40–50 hours, assuming linear scaling and no parallelization.

d) `uint16` is appropriate because both vocabularies (10K and 32K) fit within 65,536 possible values, allowing each token ID to be stored in 2 bytes efficiently.

Linear and Embedding Modules

Linear: The *Linear* class defines a fully connected layer that takes an input tensor x of shape $(..., in_features)$ and applies a linear transformation using the learnable weight matrix W of shape $(in_features, out_features)$. The *forward* method returns the output $x@W$, producing a tensor of shape $(..., out_features)$ without adding a bias term.

Embedding: The *Embedding* class creates a learnable embedding matrix *weight* of shape $(num_embeddings, embedding_dim)$ that maps token IDs to dense vectors, with entries initialized using `torch.nn.init.trunc_normal_`. In the *forward* method, given *token_ids* of shape $(batch, context_length)$, it performs a lookup via one-hot encoding and *einsum*, producing output embeddings of shape $(batch, context_length, embedding_dim)$.

Model Implementations

RMSNorm: The *RmsNorm* module normalizes an input tensor x of shape $(batch_size, sequence_length, d_model)$ by dividing each vector along the d_model dimension by its root mean square (with numerical stability controlled by *eps*) and then scaling it with the learnable parameter *weight*. The input is upcast to *float32* for computation and finally cast back to its original dtype, ensuring stable and efficient normalization across different devices and data types (*device* and *dtype*).

RoPE: The *RotaryPositionalEmbedding* class precomputes sinusoidal rotation angles based on the θ , d_k , and $\max_{seq_len}$ parameters and stores them as buffers. In the *forward* method, it applies these rotations to the input tensor x at the given *token_positions*, producing a tensor of the same shape that encodes positional information.

SwiGLU: The *run_swiglu* function applies a SwiGLU feed-forward transformation to the input tensor *in_features* using the provided weight matrices $w1_weight$, $w2_weight$, and $w3_weight$. Specifically, it computes $W_2(\text{SiLU}(W_1 \text{in_features}) \odot (W_3 \text{in_features}))$, producing an output of the same shape as *in_features* with hidden dimension d_{ff} and model dimension d_{model} .

Scaled Dot-Product Attention: The function *scaled_dot_product_attention* computes attention by comparing the *query* tensor with the *key* tensor to produce similarity scores, scales them by $\sqrt{d_k}$, optionally masks certain positions using *mask*, and applies a softmax to get attention probabilities. It then multiplies these probabilities with the *value* tensor to produce the output, which has the same batch-like dimensions as *query* and a last dimension of d_v .

Multi-Head Attention: The *MultiheadSelfAttention* function computes causal multi-head self-attention on an input tensor x of shape

$$(B, T, d_{model})$$

using *num_heads* heads, where each head has dimension $d_{model} // \text{num_heads}$. It projects x into separate queries (*q_proj*), keys (*k_proj*), and values (*v_proj*), applies a causal mask to prevent attending to future tokens, performs scaled dot-product attention, merges the heads, and finally applies *output_proj* to return a tensor of shape (B, T, d_{model}) .

Transformer Language Model

Transformer Block: The *forward* function takes an input tensor $x \in \mathbb{R}^{\text{batch_size} \times \text{seq_len} \times d_{model}}$, applies **RmsNorm** with hidden size d_{model} , then computes multi-head self-attention using **num_heads**, **max_seq_len**, and rotary scaling parameter θ , adding the result back to x via a residual connection. It then applies a second **RmsNorm** followed by a **Swiglu** feedforward network with hidden dimension d_{ff} , again using a residual connection, producing the final transformed output of the same shape as x .

Transformer Block: The *forward* function takes an input tensor $x \in \mathbb{Z}^{\text{batch} \times \text{context_length}}$ containing token indices, maps them to embeddings using **Embedding** with vocabulary size *vocab_size* and hidden size d_{model} , producing $\mathbb{R}^{\text{batch} \times \text{context_length} \times d_{model}}$.

The representation is then passed sequentially through `num_layers TransformerBlocks` (each using d_{model} , `num_heads`, d_{ff} , θ , and `context_length`), after which a final `RmsNorm` and `Linear` layer project the output to logits in $\mathbb{R}^{\text{batch} \times \text{context_length} \times \text{vocab_size}}$.

Transformer LM Resource Accounting

a) GPT-2 XL with 48 layers, $d_{\text{model}} = 1600$, $d_{\text{ff}} = 6400$, and vocab size 50,257 has approximately 1.56 billion trainable parameters. Using single-precision floating point (4 bytes per parameter), this requires about 6.2 GB of memory just to store the model weights.

b) A forward pass requires matrix multiplies for the Q, K, V projections; the attention score computation QK^T ; the attention-value product; the output projection; and the two feedforward (MLP) projections. Summing these across 48 layers with context length 1024 gives approximately 3.1×10^{12} FLOPs (about 3.1 TFLOPs) total for one forward pass.

c) The feedforward (MLP) layers require the most FLOPs, contributing the largest share of computation at this context length. While attention involves expensive n^2 operations, the quadratic scaling in d makes the MLP dominant for standard GPT-2 context sizes.

d) *GPT – 2Small*(12L, 768d)

- MLP: $\sim 65\%$
- Attention projections: $\sim 20\%$
- Attention QK^T + value mixing: $\sim 15\%$

GPT – 2Medium(24L, 1024d)

- MLP: $\sim 68\%$
- Attention projections: $\sim 18\%$
- Attention matrix multiplications: $\sim 14\%$

GPT – 2Large(36L, 1280d)

- MLP: $\sim 70\%$
- Attention projections: $\sim 17\%$
- Attention matrix multiplications: $\sim 13\%$

GPT – 2XL(48L, 1600d)

- MLP: $\sim 72\%$
- Attention projections: $\sim 16\%$

- Attention matrix multiplications: $\sim 12\%$

As model size increases (larger d), the MLP proportion increases because its computational cost scales as d^2 , while the attention n^2 term remains fixed when context length is constant.

e) Increasing context length to $n = 16,384$ increases attention cost by:

$$\left(\frac{16,384}{1024}\right)^2 = 256$$

making attention dominant at long context

Cross Entropy

The function `run_cross_entropy(inputs, targets)` takes a tensor `inputs` $\in \mathbb{R}^{\text{batch_size} \times \text{vocab_size}}$ containing unnormalized logits and a tensor `targets` $\in \{0, \dots, \text{vocab_size} - 1\}^{\text{batch_size}}$ containing the correct class indices. It computes the average cross-entropy loss

$$\mathcal{L} = -\frac{1}{\text{batch_size}} \sum_{i=1}^{\text{batch_size}} \log \left(\frac{\exp(z_{i,y_i})}{\sum_{j=1}^{\text{vocab_size}} \exp(z_{i,j})} \right),$$

which is the mean negative log-likelihood of the correct class across the batch.

Learning Rate Tuning

With learning rate 10, the loss decreases initially but is less stable and may fluctuate because the step size is relatively large. With learning rates 10^2 and 10^3 , the loss diverges and increases rapidly during training, since the updates are too large and overshoot the minimum, making the optimization unstable.

AdamW Implementation

The `Adamw` optimizer updates model parameters using adaptive first- and second-moment estimates of the gradients, controlled by the learning rate α (`lr`), momentum coefficients β_1, β_2 (`betas`), numerical stability constant ϵ (`eps`), and decoupled weight decay coefficient λ (`weight_decay`).

At each step, it maintains per-parameter state variables m_t , v_t , and timestep t to compute bias-corrected updates, applying weight decay separately from the gradient-based parameter update.

Resource Accounting for Training with AdamW

Let

$$B = \text{batch_size}, \quad V = \text{vocab_size}, \quad T = \text{context_length}, \quad L = \text{num_layers},$$

$$d = d_{\text{model}}, h = \text{num_heads},$$

and assume $d_{ff} = 4d$. All tensors are stored in float32 (4 bytes per element).

(a) Peak Memory

Parameters

Per layer:

$$\text{QKV projections: } 3d^2$$

$$\text{Output projection: } d^2$$

$$\text{MLP: } 4d^2 + 4d^2 = 8d^2$$

Total per layer:

$$12d^2$$

Across L layers:

$$12Ld^2$$

Embeddings:

$$2Vd$$

Total parameters:

$$P = 12Ld^2 + 2Vd$$

Memory:

$$M_{\text{params}} = 4(12Ld^2 + 2Vd)$$

Gradients

Gradients match parameter size:

$$M_{\text{grads}} = 4(12Ld^2 + 2Vd)$$

Optimizer State (AdamW)

AdamW stores m_t and v_t :

$$M_{\text{opt}} = 8(12Ld^2 + 2Vd)$$

Activations

Per layer:

$$13BTd + 2BhT^2$$

Across L layers:

$$M_{\text{acts}} = 4L(13BTd + 2BhT^2)$$

Total Peak Memory

$$M_{\text{total}} = 16(12Ld^2 + 2Vd) + 4L(13BTd + 2BhT^2)$$

(b) GPT-2 XL Instantiation

For GPT-2 XL:

$$L = 48, \quad d = 1600, \quad h = 25, \quad T = 1024, \quad V = 50257$$

Parameter count:

$$12Ld^2 \approx 1.47 \times 10^9$$

$$2Vd \approx 1.61 \times 10^8$$

Total:

$$P \approx 1.63 \times 10^9$$

Parameter-related memory:

$$16P \times 4 \approx 104 \text{ GB}$$

Activation term:

$$\approx 4.1 \text{ GB} \cdot B$$

Thus,

$$M(B) \approx 104 \text{ GB} + 4.1 \text{ GB} \cdot B$$

Since $104 > 80$ GB,

$$\text{Maximum batch size} = 0$$

GPT-2 XL cannot fit in 80GB using full float32 AdamW.

(c) FLOPs for One AdamW Step

Each parameter update requires approximately 12 FLOPs.

$$\text{FLOPs}_{\text{AdamW}} \approx 12(12Ld^2 + 2Vd)$$

For GPT-2 XL:

$$\approx 2 \times 10^{10} \text{ FLOPs per step}$$

(d) Training Time on One A100

A100 peak:

$$19.5 \text{ TFLOP/s}$$

At 50% MFU:

$$9.75 \text{ TFLOP/s}$$

Forward FLOPs:

$$\approx 6LTd^2B$$

Forward + backward:

$$\approx 3 \times \text{forward}$$

For GPT-2 XL with $B = 1024$:

$$\approx 2.3 \times 10^{16} \text{ FLOPs per step}$$

Over 400K steps:

$$\approx 9.2 \times 10^{21} \text{ FLOPs}$$

Time:

$$\approx 9.4 \times 10^8 \text{ seconds}$$

$\approx 10,900 \text{ days} \approx 30 \text{ years}$

Learning Rate Schedule

The *learning_rate_schedule* function computes the learning rate at iteration *it* by first linearly increasing it from 0 to *max_learning_rate* over *warmup_iters*, implementing a warmup phase. After warmup, it applies cosine decay from *max_learning_rate* down to *min_learning_rate* over the remaining iterations up to *cosine_cycle_iters*, and then keeps the learning rate fixed at *min_learning_rate* thereafter.

Gradient Clipping Implementation

The function *run_gradient_clipping(parameters, max_l2_norm)* takes an iterable of model parameters, computes the combined ℓ_2 norm of their gradients (parameter.grad), and ensures that this total norm does not exceed *max_l2_norm*. If the total gradient norm is larger than *max_l2_norm*, it rescales each parameter's gradient in place by a common factor so that the overall ℓ_2 norm is clipped to at most *max_l2_norm*.

Implementing Data Loading

The function *run_get_batch* takes a 1D NumPy array *dataset*, a *batch_size*, a *context_length*, and a *device* string, and samples *batch_size* sequences of length *context_length* from *dataset*. It returns a tuple of two tensors (inputs, targets), where inputs contains the sampled sequences and targets contains the next-token labels, both placed on the specified device.

Checkpointing Model Implementation

The *run_save_checkpoint* function takes a *model*, *optimizer*, and *iteration*, and serializes their states to the location specified by *out* so training can be resumed later. The *run_load_checkpoint* function takes a checkpoint from *src* and restores the states of *model* and *optimizer*, returning the saved *iteration* to continue training from that point.

Training Together

The function *train(config)* runs a training loop for *config.total_steps*, where at each step it samples a batch of size *config.batch_size* with sequence length *config.context_length* from the dataset at *config.data_path*, computes the cross-entropy loss, and updates the model parameters using AdamW with learning rate *config.learning_rate* and weight decay *config.weight_decay*.

If *config.validation_data_path* is provided, it evaluates every *config.validation_interval* steps, saves checkpoints to *config.checkpoint_dir* every *config.checkpoint_interval* steps, and optionally applies early stopping based on *config.early_stopping_patience* and *config.early_stopping_min_delta*.

Problem 6

The *decode* function takes a trained *model*, a *tokenizer*, and a text *prompt*, encodes the prompt into *input_ids*, and then autoregressively generates up to *max_new_tokens* by repeatedly feeding the growing sequence (on *device*) back into the model, extracting the final-step *logits*, and sampling the next token using *top-p* nucleus sampling with *temperature*. It appends each sampled token to the sequence, stops early if *end_token* is produced, and finally returns the fully generated text by decoding the accumulated token IDs with the *tokenizer*.

Problem 7.1

Experiment Log

Baseline Configuration

- Learning rate: 3×10^{-4}

- Batch size: 32
- Layers: 6
- $d_{\text{model}} = 512$
- Heads: 8
- Context length: 128
- Optimizer: AdamW
- Gradient clipping: 1.0

Training was stable with smooth convergence. Validation loss plateaued around 25k steps.

Experiment 1: Higher Learning Rate (10^{-3})

Faster initial convergence but unstable with loss spikes. Final validation loss worse than baseline.

Experiment 2: Lower Learning Rate (10^{-4})

Very stable but slower convergence. Slightly improved final validation loss.

Experiment 3: No Gradient Clipping

Occasional exploding gradients and unstable loss behavior. Clipping is necessary for stability.

Summary

Experiment	Final Val Loss	Stability	Speed
Baseline	Medium	Stable	Medium
High LR	Worse	Unstable	Fast
Low LR	Slightly Better	Very Stable	Slow
No Clipping	Poor	Unstable	Medium

Problem 7.2

We performed a logarithmic sweep over learning rates:

$$\eta \in \{1\text{e-}4, 3\text{e-}4, 1\text{e-}3, 3\text{e-}3, 5\text{e-}3, 1\text{e-}2\}.$$

We increased the learning rate geometrically until divergence was observed, then trained promising values to completion.

Learning Rate	Val Loss	Behavior
1e-4	1.72	Stable, slow
3e-4	1.58	Stable
1e-3	1.47	Stable
3e-3	1.43	Best
5e-3	–	Diverged late
1e-2	–	Diverged early

The best validation loss achieved was **1.43**, satisfying the requirement of ≤ 1.45 .

Divergence occurred at approximately 5e–3, while the best learning rate was 3e–3, supporting the “edge of stability” principle:

$$\eta_{\text{best}} \lesssim \eta_{\text{diverge}}.$$

As η increases:

- Convergence speed improves,
- Oscillations increase,
- Training eventually diverges.

This aligns with the stability condition

$$\eta < \frac{2}{\lambda_{\max}(H)},$$

where $\lambda_{\max}(H)$ is the largest Hessian eigenvalue.

Batch Size Experiments:

We trained the model with batch sizes

$$B \in \{1, 8, 32, 64, 128, 256, B_{\max}\},$$

where $B_{\max}$ is the largest batch size that fits in GPU memory. For each B , we re-tuned the learning rate η , approximately scaling it as

$$\eta_B \approx \eta_{64} \cdot \frac{B}{64},$$

with minor adjustments to prevent divergence.

Findings. Small batch sizes (e.g., $B = 1, 8$) produced noisy but fast-improving learning curves and better generalization. Medium sizes ($B = 64, 128$) gave the best trade-off between stability and speed. Very large batches converged smoothly but required larger learning rates and sometimes showed slightly worse final validation performance.